

Радар на Arduino с визуализацией на PyQt

Этот проект представляет собой радар, основанный на Arduino, который собирает данные о расстоянии до объектов и визуализирует их с помощью PyQt.

Проект состоит из двух частей: скетча для Arduino и приложения на Python с использованием библиотеки PyQt.

Оборудование

- Плата Arduino (например, Arduino Uno)
- Ультразвуковой датчик HC-SR04
- Сервопривод MG90 или аналог
- Соединительные провода

Схема подключения

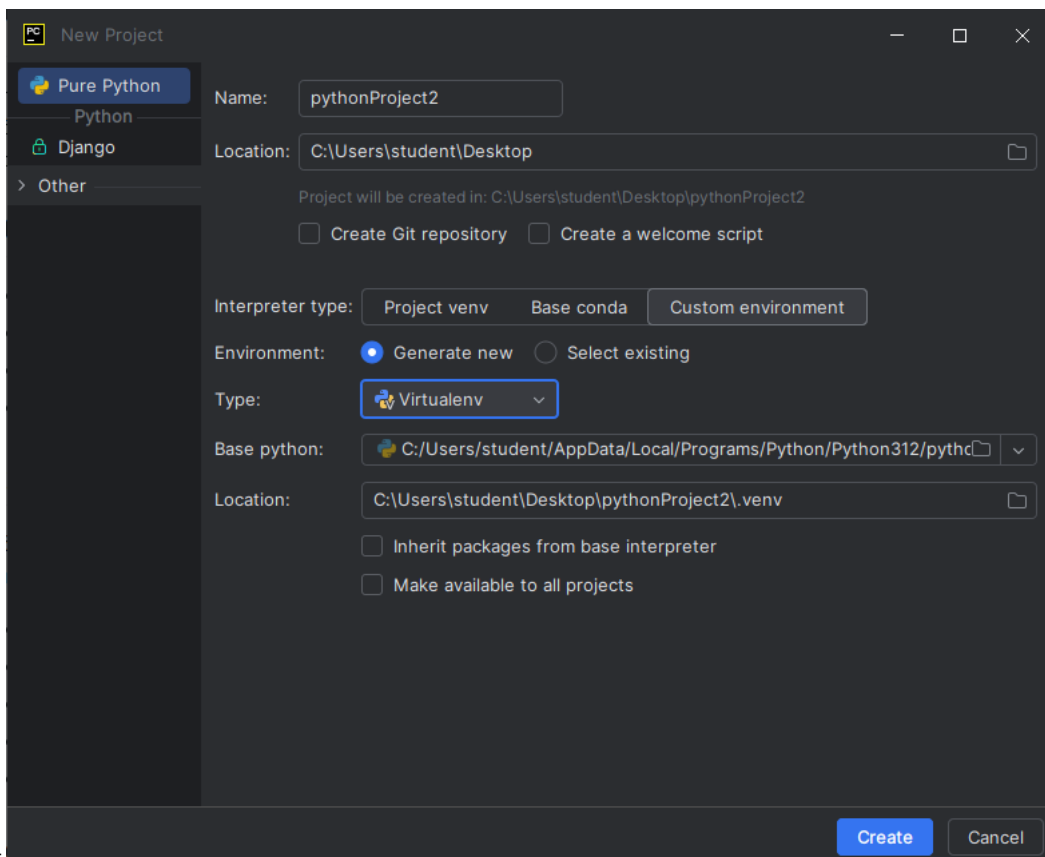
Убедитесь, что ваш ультразвуковой датчик и сервопривод правильно подключены к Arduino согласно следующей схеме:

- VCC к 5V на Arduino
- GND к GND на Arduino
- ECHO к цифровому пину 2
- TRIG к цифровому пину 3
- Сервопривод к цифровому пину 4

Создание программы

Python и PyQt

1. Убедитесь, что у вас установлен [Python 3.6 или выше](#).
2. Установите [Pycharm CE](#) *на сайте пролистайте чуть ниже.



3. Создайте новый проект
4. Поздравляю, можно начать программировать!

- Установим все библиотеки

```
import serial # Библиотека Python для работы с последовательными соединениями
import serial.tools.list_ports # Инструменты для перечисления доступных последовательных портов
import pyqtgraph as pg # Библиотека для создания интерактивных графиков
import numpy as np # Библиотека для численных операций
from PyQt6.QtWidgets import QApplication, QMainWindow, QMessageBox # Компоненты для GUI
from PyQt6.QtSerialPort import QSerialPort # Обработка последовательного порта для PyQt
from design import Ui_MainWindow # Импорт класса дизайна UI
```

- Создадим структуру класса

```
class SerialApp(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self) # Настройка UI компонентов
        self.serial = QSerialPort() # Создание экземпляра последовательного порта
        self.setWindowTitle("Radar") # Установка заголовка окна

    def populate_ports(self):
        return

    def connect_serial(self):
        return

    def disconnect_serial(self):
        return

    def read_serial_port(self):
        return

    def setup_radar_graph(self):
        return

    def update_radar(self):
        return

# Основной блок для запуска приложения
if __name__ == "__main__":
    app = QApplication(sys.argv) # Создание приложения
    window = SerialApp() # Создание экземпляра SerialApp
    window.show() # Показать основное окно
    sys.exit(app.exec()) # Выполнение приложения
```

- Разберем каждую функцию

-

- Функция `populate_ports(self)` обновляет список подключенных портов. ВАЖНО! Список обновляется только при запуске программы!

```
def populate_ports(self):
    self.portlist.clear() # Очистка существующего списка портов
    ports = serial.tools.list_ports.comports() # Перечисление всех доступных последовательных портов
    for port in ports:
        self.portlist.addItem(port.device) # Добавление каждого порта в выпадающий список
```

-

- Функция `connect_serial(self)` открывает последовательный порт для дальнейшей работы с Arduino (Важно: при подключении к порту, вы занимаете так называемый "трафик" - другое приложение не сможет подключиться к этому порту важно закрывать порт в Arduino IDE перед подключением из приложения)

```
def connect_serial(self):
    selected_port = self.portlist.currentText() # Получение текущего выбранного порта
    if not selected_port:
        QMessageBox.warning(self, "Error", "No port selected!") # Предупреждение, если порт не выбран
```

```

        return

self.serial.setPortName(selected_port) # Установка имени выбранного порта
self.serial.setBaudRate(9600) # Установка скорости передачи данных

# Попытка открыть последовательный порт
if self.serial.open(QSerialPort.OpenModeFlag.ReadWrite):
    QMessageBox.information(self, "Success",
                            f"Connected to {selected_port}") # Сообщение об успешном подключении
else:
    QMessageBox.critical(self, "Error",
                        "Failed to open serial port") # Сообщение об ошибке, если не удалось открыть

```

-
- Функция `disconnect_serial(self)` закрывает последовательный порт (отключает от Arduino)

```

def disconnect_serial(self):
    if self.serial.isOpen(): # Проверка, открыт ли порт
        self.serial.close() # Закрытие последовательного порта
        QMessageBox.information(self, "Disconnected", "Serial port closed.") # Уведомление пользователя

```

-
- Функция `read_serial_port(self) [text]`

```

def read_serial_port(self):
    # Проверка, есть ли данные для чтения
    while self.serial.canReadLine():
        rx = self.serial.readLine() # Чтение строки данных
        rxs = str(rx, 'utf-8').strip() # Декодирование байтов в строку и удаление пробелов
        data = rxs.split(',') # Разделение строки на компоненты

        # Проверка формата данных

        if data[0] == '1':
            angle = int(data[1]) # Преобразование угла из строки в целое число
            distance = float(data[2]) # Преобразование расстояния из строки в число с плавающей точкой

            self.pos_1.setText(str(angle) + "°") # Обновление метки угла в UI
            self.dist_1.setText(str(distance) + " cm") # Обновление метки расстояния в UI

            self.update_radar(angle, distance) # Обновление графика радара новыми данными

```

-
- Graph

```

def setup_radar_graph(self):
    self.radar_plot = pg.PlotWidget() # Создание виджета графика
    self.gridLayout.addWidget(self.radar_plot, 0, 0, 1, 1) # Добавление виджета графика в сеточный макет

    self.radar_plot.setAspectLocked(True) # Блокировка соотношения сторон для сохранения круговой формы ра;
    self.radar_plot.setXRange(-self.max_distance, self.max_distance) # Установка диапазона по оси X
    self.radar_plot.setYRange(0, self.max_distance) # Установка диапазона по оси Y

    self.radar_plot.getAxis('bottom').setLabel('Distance (cm)') # Подпись для оси X
    self.radar_plot.getAxis('left').setLabel('Angle') # Подпись для оси Y
    self.radar_plot.showGrid(x=True, y=True) # Отображение сетки на графике

    # Инициализация пустого графика с точками, представленными зелеными кругами
    self.radar_curve = self.radar_plot.plot([], [], pen=None, symbol='o', symbolSize=5, symbolBrush=(0, 255

def update_radar(self, angle, distance):
    # Вычисление координат в декартовой системе для построения
    x = distance * np.cos(np.radians(angle)) # X-координата на основе угла и расстояния
    y = distance * np.sin(np.radians(angle)) # Y-координата на основе угла и расстояния

    self.angle_data.append(x) # Добавление X-координаты в список данных угла
    self.distance_data.append(y) # Добавление Y-координаты в список данных расстояния

```

```

# Ограничение размера списков данных до 180 элементов для поддержания производительности
if len(self.angle_data) > 180:
    self.angle_data.pop(0) # Удаление старой точки данных по X
    self.distance_data.pop(0) # Удаление старой точки данных по Y

# Обновление графика радара с новыми данными
self.radar_curve.setData(self.angle_data, self.distance_data)

```

Arduino

1. Установите [Arduino IDE](#).
2. Установите драйвер [CH340](#).
3. Код

- Подключение библиотек и создание переменных

```

#include <Servo.h> // Подключаем библиотеку для работы с сервоприводом
Servo myservo;    // Создаем объект myservo для управления сервоприводом

#define HC_TRIG 3  // Определяем пин для TRIG ультразвукового датчика
#define HC_ECHO 2  // Определяем пин для ECHO ультразвукового датчика

float distFilt = 0; // Переменная для хранения отфильтрованного расстояния
int pos = 23;       // Начальная позиция сервопривода (в градусах)
bool direction = true; // Флаг направления движения сервопривода (true - против часовой, false - по часовой)

```

- Инициализация

```

void setup() {
    Serial.begin(9600); // Инициализация последовательного порта со скоростью 9600 бод
    pinMode(HC_TRIG, OUTPUT); // Устанавливаем TRIG как выход
    pinMode(HC_ECHO, INPUT);  // Устанавливаем ECHO как вход
    myservo.attach(4);        // Подключаем сервопривод к пину 4
    delay(500);               // Небольшая задержка для стабилизации
}

```

- Основной цикл программы

```

void loop() {
    float dist = getDist(); // Получаем необработанное расстояние с датчика
    distFilt += (dist - distFilt) * 0.2; // Применяем фильтрацию (экспоненциальное сглаживание)

    // Двигаем сервопривод в пределах от 0 до 180 градусов
    if (direction && pos <= 180) {
        pos++; // Увеличиваем угол поворота
        myservo.write(pos); // Устанавливаем новую позицию сервопривода
    } else if (pos >= 0) {
        pos--; // Уменьшаем угол поворота
        direction = false; // Меняем направление движения
        myservo.write(pos); // Устанавливаем новую позицию сервопривода
    } else {
        direction = true; // Возвращаем направление в исходное
    }

    // Отправляем данные через Serial в формате "1,угол,расстояние"
    Serial.print("1,");
    Serial.print(pos);
    Serial.print(",");
    Serial.println(distFilt);

    delay(50); // Небольшая задержка для плавности работы
}

```

- Функция для получения расстояния

```
// Функция получения расстояния с ультразвукового датчика
float getDist() {
digitalWrite(HC_TRIG, HIGH); // Посылаем короткий импульс 10 мкс на TRIG
delayMicroseconds(10);      // Ожидание 10 микросекунд
digitalWrite(HC_TRIG, LOW);  // Завершаем импульс

uint32_t us = pulseIn(HC_ECHO, HIGH); // Измеряем длительность импульса на ECHO

return (us / 58.3); // Преобразуем время в микросекундах в расстояние в сантиметрах
}
```

4. Загрузка на микроконтроллер

Выбираем свою плату. Если это Arduino Nano, выбираем в Инструменты\Плата\Arduino Nano. Если другая – выбираем другую. Нажимаем стрелочку в левом верхнем углу (загрузить прошивку).

[Для Arduino Nano] В микроконтроллер китайских нано защит “старый” загрузчик, поэтому выбираем Инструменты\Процессор\ATmega328p (Old Bootloader). Некоторые китайцы зашивают в свои платы новый загрузчик, поэтому если прошивка не загрузилась (загрузка идёт минуту и вылетает ошибка avrdude: stk500_getsync()) – попробуйте сменить пункт Процессор на ATmega328p.

Если появилась надпись “Загрузка завершена” – значит всё в порядке и можно прошивать другие скетчи.

Запуск приложения

1. Подключите Arduino к компьютеру через USB-кабель.
2. Убедитесь, что вы знаете, какой COM-порт использует Arduino (например, /dev/ttyUSB0 на Linux/Mac или COM3 на Windows).
3. Убедитесь, что Arduino скетч загружен.
4. Запустите приложение PyQt:

```
radar_pyqtgraph.py
```

5. Введите COM-порт в диалоговом окне и нажмите “Подключиться”.
6. Вы должны увидеть визуализацию данных радара в реальном времени.



Лицензия

Этот проект лицензирован под [MIT License](#).

Контакты

Если у вас есть вопросы, вы можете связаться со мной по [электронной почте](#).